TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN
Môn học: **CSC10043 - NHẬP MÔN TRÍ TUỆ NHÂN TẠO**
Giảng viên: **TS. Nguyễn Văn X** | Lớp: **21CNTN**

TÊN ĐỀ TÀI / TÊN BÀI TẬP

BÁO CÁO ĐỒ ÁN MÔN HỌC / BÀI TẬP CÁ NHÂN

Họ và tên sinh viên: Nguyễn Văn A, Trần Thị B | **Năm học:** Học kỳ I, 2026 - 2027

1. Mô tả bài toán và Yêu cầu

1.1. Yêu cầu đề bài

Sinh viên ghi lại đề bài hoặc yêu cầu của bài tập/đồ án tại đây.

- **Mục tiêu:** Xây dựng chương trình giải quyết bài toán…
- **Đầu vào (Input):** Tệp dữ liệu hoặc thông số cấu hình đầu vào.
- **Đầu ra (Output):** Kết quả tính toán, dự đoán hoặc báo cáo xử lý.

1.2. Môi trường thực thi

Chương trình được cài đặt và kiểm thử trên môi trường:

- **Ngôn ngữ:** Python 3.10+ (hoặc C++17, Java).
- **Thư viện chính:** `numpy`, `matplotlib`.
- **Cách chạy:** Chạy lệnh `python main.py` trên dòng lệnh.

2. Phương pháp và Thiết kế giải thuật

2.1. Ý tưởng giải quyết

Trình bày tóm tắt ý tưởng hoặc phương pháp tiếp cận chính để giải quyết bài toán:

1. **Bước 1 - Đọc và xử lý dữ liệu:** Kiểm tra tính hợp lệ của dữ liệu đầu vào.

2. **Bước 2 - Xử lý tính toán:** Áp dụng giải thuật để tìm lời giải tối ưu.
3. **Bước 3 - Xuất kết quả:** Hiển thị kết quả ra màn hình hoặc ghi vào tệp.

2.2. Công thức tính toán (nếu có)

Nếu bài toán có công thức toán học, sinh viên trình bày như sau:

$$f(x) = \sum_{i=1}^{n} w_i x_i + b \tag{1}$$

Trong đó w_i là trọng số, x_i là giá trị đầu vào và b là hệ số điều chỉnh.

3. Cài đặt và Mã nguồn chính

3.1. Mã giả giải thuật

Trình bày các bước thực hiện của thuật toán chính:

Thuật toán 1: Thuật toán tìm kiếm phần tử

Đầu vào: Mảng đã sắp xếp A, giá trị cần tìm x.
Đầu ra: Vị trí của x trong mảng, hoặc -1 nếu không tìm thấy.

```
l ← 0, r ← |A| − 1
while l ≤ r do
    m ← ⌊(l + r)/2⌋
    if A[m] = x then
        return m
    else if A[m] < x then
        l ← m + 1
    else
        r ← m − 1
    end if
end while
return −1
```

3.2. Đoạn mã nguồn cốt lõi

Sinh viên chèn đoạn mã nguồn chính quan trọng nhất (tránh chèn toàn bộ mã nguồn quá dài):

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Listing 1: Hàm tìm kiếm nhị phân bằng Python

Lưu ý khi nộp bài

Mã nguồn nộp kèm cần có chú thích (comment) rõ ràng ở các hàm chính và tuân thủ chuẩn đặt tên biến.

4. Kết quả thực nghiệm và Tự đánh giá

4.1. Kết quả kiểm thử trên các bộ dữ liệu

Tóm tắt kết quả chạy thử nghiệm chương trình trên các trường hợp kiểm thử (test cases):

Bảng 1: Bảng kết quả kiểm thử chương trình

Test Case	Mô tả dữ liệu thử nghiệm	Kết quả mong đợi	Đánh giá
01	Dữ liệu mảng rỗng $A = []$	Trả về -1	Đạt
02	Phần tử nằm ở đầu mảng	Trả về vị trí 0	Đạt
03	Phần tử không tồn tại trong mảng	Trả về -1	Đạt
04	Mảng lớn có 10^6 phần tử	Thời gian < 0.1 s	Đạt

4.2. Bảng tự đánh giá mức độ hoàn thành

Sinh viên tự đánh giá tỷ lệ hoàn thành các yêu cầu của bài tập:

Bảng 2: Tự đánh giá mức độ hoàn thành yêu cầu

STT	Yêu cầu của đề bài	Mức độ hoàn thành
1	Cài đặt đúng thuật toán và cấu trúc dữ liệu yêu cầu	100%
2	Xử lý đầy đủ các trường hợp ngoại lệ (Edge cases)	100%
3	Tối ưu hóa thời gian thực thi và bộ nhớ sử dụng	100%
4	Viết báo cáo rõ ràng, mạch lạc đúng chuẩn quy định	100%

4.3. Kết luận

Chương trình đã hoàn thành đầy đủ các yêu cầu đề ra, chạy ổn định và vượt qua toàn bộ các bộ dữ liệu kiểm thử.

Tài liệu tham khảo